CS 166 ASSIGNMENT 3 SOLUTION

Overview

- Marks on question:
  - No mark: 1-part short answer question (the key point(s) for explanation is underlined)
  - !: purely auto-graded question (answer boxed or color-coded)
  - +: muli-part question with auto-graded parts and explanation parts.
    The answers for auto-graded parts are boxed, the key point(s) for explanation is underlined.
- Grading rules:
  - Auto-graded questions are all-or-nothing
  - 0.5 pt or 1 pt short answers are all-or-nothing
  - Other short answers questions: -0.5 for each key point missing.
  - If your answer is totally wrong, you will get 0.5 points (on whole question, not each sub-question).

Week 5 - Software Flaws & Malware (Q1 - Q8)

!Q1: Malware timeline (L10 P7 - 10)

All correct. Summarized on L11 P2.

!Q2: Malware detection (L10 P11 - 14)

See quiz answers. Summarized on L11 P3.

!Q3: Evade signature detection (L10 P15 - 17)

See quiz answers. Summarized on L11 P4.

+Q4: Canary to detect buffer overflow (L9 P15)

I. When the canary is overridden, it means the buffer overflowed and may override the return address.

II. True, Microsoft's implantation is flawed. Attacker can also specify the handler to execute the "evil code".

Q5: Defending stack-based buffer overflow attacks (L9 P15 & P16)

I. (Mention at least 2) Non-executable stack; ASLR; safer languages (Java, C#); safer C functions

II. (Explain the 2 ways mentioned in I.)

- Use non-executable stack so Trudy can't execute her evil code by overflowing the stack to redirect the return address.
- Use ASLR to randomize place where code loaded in memory so Trudy can't easily find the address to redirect to.
- Use safe languages/safer C functions that stop the program if buffer overflowed.

Yan Chen
Computer Science Department

+Q6: Incomplete mediation (L9 P17 & P18)

I. If input is not validated, it is called incomplete mediation.

II. False, need also validate on server side since attacker can change the input after client side verifies the input. For the example, you can make up your own, or just use the one given in L9 P18.

Q7: Race condition attack example (L9 P19)

I. Race condition attack, since reading (step 2) & writing (step 5) data are 2 separate "stages".

II. Various attack possible, and the curial step always happens between step 2 and step 4/5.
For example, put 2 cards (let's say card A has $500 and card B has $1) on top of each other in step 1, so the machine reads card A (i.e., x = 500) in step 2. Insert a $5 bill in step 3 so y = 5.
Then before pressing ENTER in step 4, take out card A but leave card B. Then machine will write x + y = $505 to card B in step 5! Card A still has $500, and card B now has $505 although you only added $5!

Q8: Heap-based buffer overflow example (L9 P11)

I. The following is printed: BEFORE: buf2 = 22222222 AFTER: buf2 = 11122222

II. The content of buf2 changed since buf1 is overflowed by 3, which overrides the first 3 chars of buf2.

Week 6 - Other Attacks on Software (Q8 - Q12)

Q9: Mitigating an SRE Attack (L11 P13 - 16)

I. (Mention at least 3) Anti-disassembly; anti-debugging; tamper-resistance; code obfuscation

II. (Explain the 3 ways mentioned in I.)

- Anti-disassembly to confuse static view of code. E.g., encrypt object code or put some junk assembly instructions, etc.
- Anti-debugging to confuse dynamic view of code. E.g., check IsDebuggerPresent or monitor use of debug registers, etc.
- Tamper-resistance makes patching harder by detecting if the code is being changed through hash value.
- Code obfuscation means making code hard to understand so it takes the attacker more time to analyze.

Note that writing "anti-D's" is consider as wrong answer as explained in class.

+Q10: Linearization attack based on timing (L12 P7 - 9)

I. False, the code can't prevent a linearization attack based on timing. There is still time difference between correct and incorrect case: if a character is wrong, flag = false will be executed, which may take time. In this case, an incorrect serial number takes more time to check than a correct one.

II. If doesn't use linearization attack, need $32^8 / 2 = 2^{39}$ tries on average (note that $32^8 / 2 \neq 32^7$);
if use linearization attack based on time, need 32 * 8 / 2 = 27 tries on average.
(or worst case without / 2)

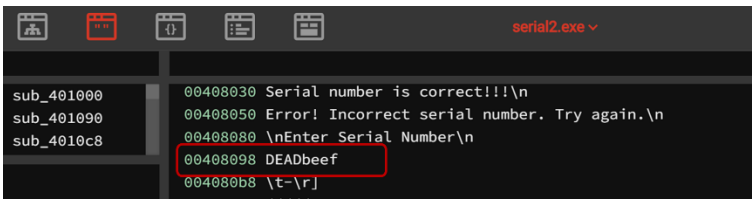
!Q11 Try it yourself - linearization attack based on timing (L12 P7 - 9)

Recover one character at a time by finding the one that takes the longest time (or disassemble the exe to get the number). The correct number is: 315279.

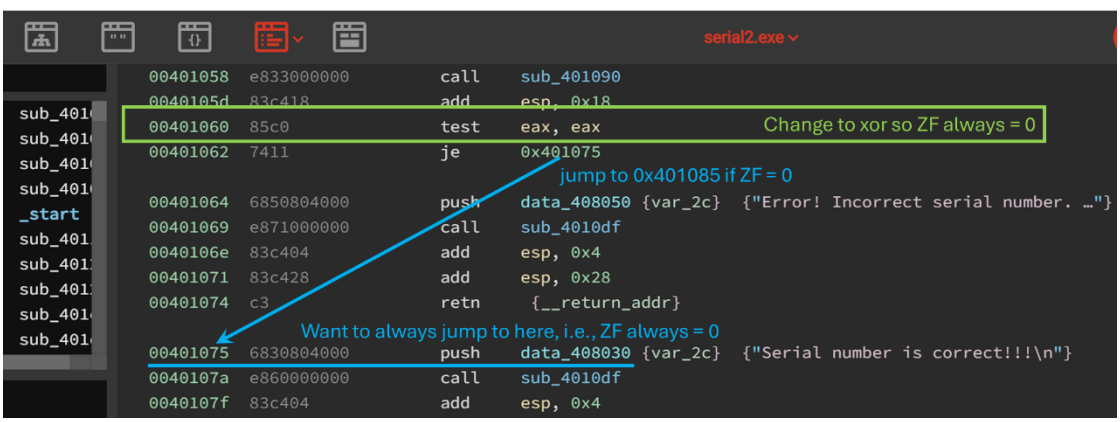
+Q12 Try it yourself - simple SRE (L11 P10 - 12)

You can use whatever disassembler and hex editor you like. The screenshots here are from the online tools demoed in class.

I. Once disassembled, check strings around "Enter Serial Number". The serial number is DEADbeef.



II. Similar to what we did in class, we want the program to always jump to the address of "Serial number is correct" for all inputs. As we can see, the address is data_408030.

Checking the assembly code, we see 00401075 will push data_408030 and print it.
In order to jump to 00401075, the zero flag needs to be 0 (je means jump if zero flag = 0).

Therefore, change `test eax, eax` (85 c0 74 11 68) to `xor eax, eax` (31 c0 74 11 68) in any hex editor you like, and save the new .exe. The new .exe will show "Serial number is correct" for all inputs.